

Répartition des tâches – EasySave v3.0

Branche de base : `develop` → `release/v3.0` au moment de la stabilisation.

Vue d'ensemble

Dev	Domaine
Dev 1	Parallélisme, ordonnancement, priorité, gros fichiers, UML
Dev 2	Contrôle des travaux (Pause/Stop/Play), logiciel métier, CryptoSoft mono-instance
Dev 3	Paramètres généraux UI, Docker log centralisation, documentation

Dev 1 – Parallélisme, ordonnancement et UML

Branches Git Flow :

```

|                               |                                          |
|-------------------------------|------------------------------------------|
| feature/v3-parallel-execution | (base : develop)                         |
| feature/v3-priority-files     | (base : feature/v3-parallel-execution)   |
| feature/v3-large-file-mutex   | (base : feature/v3-parallel-execution)   |
| feature/v3-uml-docs           | (base : develop, à finaliser en dernier) |

```

T1 – Parallélisation des sauvegardes (`feature/v3-parallel-execution`)

****Fichiers :**** `BackupManager.cs`, `BackupStrategyBase.cs`,
`BackupState.cs`

- Remplacer `RunAll()` séquentiel par des `Task` lancés en parallèle, un par travail.
- Passer un `CancellationToken` à chaque stratégie (prérequis pour le Stop de Dev 2).
- Rendre les accès à `BackupState` thread-safe (`lock` ou `Interlocked` sur les compteurs).
- Exposer `CancellationTokenSource` par travail dans `BackupManager` (Dev 2 s'en servira).
- Tests : vérifier que deux travaux s'exécutent bien en parallèle (chronométrage).

> ****< Synchro T1 → T4 (Dev 2) :**** avant de merger, aligner avec Dev 2 sur la signature exacte de `CancellationTokenSource` exposée par `BackupManager` – Dev 2 en a besoin pour implémenter Stop.

>

> ****< Synchro T1 ↔ T4 (avant de brancher) :> **** définir ensemble l'enum ``PlaybackState { Running, Paused, Stopped }`` et l'ajouter dans ``BackupState.cs`` sur ``develop`` dans une PR commune, avant que T1 et T4 ne branchent chacun de leur côté.

T2 – Gestion des fichiers prioritaires (``feature/v3-priority-files``)

****Fichiers :> **** ``AppConfig.cs``, ``BackupStrategyBase.cs``, ``ConfigManager.cs``

- Ajouter ``List<string> PriorityExtensions`` dans ``AppConfig``.
- Barrière partagée (``SemaphoreSlim`` ou ``ManualResetEventSlim``) : tant qu'un fichier prioritaire est en attente sur n'importe quel travail, les fichiers non-prioritaires attendent.
- Vérification dans ``BackupStrategyBase.CopyFile()`` avant chaque copie.
- Tests : scénario ``.pdf`` (prioritaire) + ``.txt`` (non-prioritaire) en parallèle.

> ****< Synchro T2 → Dev 3 :> **** une fois ``PriorityExtensions`` stable dans ``AppConfig``, en informer Dev 3 – il expose le champ dans les paramètres généraux UI et Dev 1 le documente dans l'UML.

T3 – Interdiction de transferts simultanés de gros fichiers (``feature/v3-large-file-mutex``)

****Fichiers :> **** ``AppConfig.cs``, ``BackupStrategyBase.cs``

- Ajouter ``int LargeFileSizeKb`` dans ``AppConfig``.
- Mutex statique partagé : un seul fichier > seuil transféré à la fois ; les autres travaux continuent avec leurs petits fichiers.
- Tests : deux fichiers volumineux ne doivent pas se transférer simultanément.

> ****< Synchro T3 → Dev 3 :> **** même chose que T2 – signaler à Dev 3 quand ``LargeFileSizeKb`` est stable pour qu'il l'expose dans les paramètres généraux.

T4 – UML v3.0 (``feature/v3-uml-docs``)

****Fichiers :> **** ``diagrams.md``

- Mettre à jour le ****diagramme de classes**** : ``PlaybackState``, ``PriorityExtensions``, ``LargeFileSizeKb``, ``LogDestination``, ``LogForwarder``, mutex CryptoSoft.
- Mettre à jour le ****diagramme de séquence**** ``run backup job`` : parallélisme, Pause/Stop, barrière prioritaire, mutex gros fichiers.
- Ajouter un ****diagramme de séquence**** pour la centralisation Docker.
- Mettre à jour le ****diagramme d'activité**** : parallélisme, pause logiciel métier, barrière.

> T4 se fait ****en dernier****, une fois que Dev 1, Dev 2 et Dev 3 ont mergé leurs features dans `develop`.

Dev 2 – Contrôle des travaux et CryptoSoft

Branches Git Flow :

```

```
feature/v3-play-pause-stop (base : develop)
feature/v3-business-software-pause (base : feature/v3-play-pause-stop)
feature/v3-cryptosoft-singleton (base : develop)
```
```

T5 – Play / Pause / Stop par travail (`feature/v3-play-pause-stop`)

****Fichiers :**** `BackupManager.cs`, `BackupStrategyBase.cs`,
`BackupState.cs`, ViewModel WPF

- Ajouter `enum PlaybackState { Running, Paused, Stopped }` dans `BackupState`.
- Pause effective ****après le fichier en cours**** : vérification dans `CopyFile()` à chaque itération via un `ManualResetEventSlim` par travail.
- Stop immédiat : lever le `CancellationToken` fourni par Dev 1.
- Exposer `PauseJob(int)`, `ResumeJob(int)`, `StopJob(int)` dans `BackupManager`.
- Suivi temps réel : `BackupState.Progression` rafraîchi dans le ViewModel via `DispatcherTimer` (~500 ms).
- Tests : Pause → état suspendu, Resume → reprise, Stop → arrêt propre.

> ****< Synchro T5 → T1 (Dev 1) :**** T5 ne peut pas être finalisée sans que `feature/v3-parallel-execution` soit mergée dans `develop` – attendre ou rebasez dessus pour accéder au `CancellationTokenSource`.

T6 – Pause automatique sur logiciel métier (`feature/v3-business-software-pause`)

****Fichiers :**** `BackupManager.cs`, `ProcessDetector.cs`

- En v2.0 la détection bloquait le lancement. En v3.0 : pause de ****tous les travaux actifs**** à la détection, reprise automatique à la disparition.
- `Timer` dans `BackupManager` qui sonde `IBusinessSoftwareDetector` toutes les secondes.
- À la détection : `PauseJob()` sur tous les actifs ; à la disparition : `ResumeJob()`.
- Consigner pause et reprise dans le log.
- Tests : mock `IBusinessSoftwareDetector` → travaux mis en pause puis repris.

> ****> Synchro T6 → T5 (Dev 2 interne) : **** T6 appelle `PauseJob()` / `ResumeJob()` – T5 doit être mergée avant de finaliser T6.

T7 – CryptoSoft mono-instance (`feature/v3-cryptosoft-singleton`)

****Fichiers : **** `CryptoSoftRunner.cs`, `ICryptoService.cs`

- `SemaphoreSlim(1,1)` statique dans `CryptoSoftRunner` : un seul appel à `cryptosoft.exe` à la fois.
- Optionnel : `Mutex` nommé système (`"Global\\CryptoSoftMutex"`) pour protection multi-process.
- Tests : deux appels simultanés → le second attend la fin du premier.

> ****> Synchro T7 → T4 (Dev 1) : **** aucun changement d'interface publique – signaler à Dev 1 que le Mutex est en place pour qu'il le note dans le diagramme de classes.

Dev 3 – Paramètres généraux, Docker et documentation

> Les tâches T8 et T9 sont indépendantes du code métier de Dev 1 et Dev 2 et peuvent démarrer immédiatement. T10 attend la stabilisation du code.

****Branches Git Flow : ****

\\`

feature/v3-settings-ui	(base : develop)
feature/v3-docker-log-centralization	(base : develop)
feature/v3-release-docs	(base : develop, à finaliser en dernier)

\\`

T8 – Paramètres généraux UI (`feature/v3-settings-ui`)

****Fichiers : **** ViewModel WPF settings, `ConsoleMenu.cs` (si fallback console), `AppConfig.cs`

- Exposer dans l'interface les nouveaux paramètres configurables :
 - liste des extensions prioritaires (`PriorityExtensions`)
 - logiciel métier (`BusinessSoftwareName`)
 - seuil des fichiers volumineux (`LargeFileSizeKb`)
 - destination des logs (`LogDestination` : local / remote / both)
- Persister les changements via `ConfigManager.Instance.Save()`.
- Tests : vérifier que les valeurs sont bien persistées et rechargées au démarrage.

> ****> Synchro T8 → Dev 1 (T2, T3) : **** attendre que `PriorityExtensions` et `LargeFileSizeKb` soient stables dans `AppConfig` avant de brancher les bindings – coordonner avec Dev 1.

>

> ****> Synchro T8 → Dev 2 (T6) : **** `BusinessSoftwareName` est déjà dans

`AppConfig` depuis v2.0 ; vérifier avec Dev 2 s'il y a des changements de comportement à refléter dans l'UI.

T9 – Centralisation des logs via Docker (`feature/v3-docker-log-centralization`)

****Fichiers :**** nouveau `LogForwarder.cs`, `Logger.cs`, `AppConfig.cs`, nouveau `docker-compose.yml`

- Créer `LogForwarder` : envoie chaque `LogEntry` sérialisée au service Docker via `HttpClient` POST JSON (fire-and-forget, ne bloque pas la sauvegarde).
- Dans `Logger.Log()` : appeler `LogForwarder` si `Remote` ou `Both`, continuer à écrire localement si `Local` ou `Both`.
- Service Docker : conteneur léger qui reçoit les entrées et les écrit dans un fichier journalier unique côté serveur, avec un champ `Machine` dans le payload pour différencier les sources.
- Mode dégradé : si le serveur est inaccessible, le log local est toujours garanti (try/catch autour du forward).
- Tests : log local écrit même quand le forward échoue.

> ****> Synchro T9 → Dev 1 + Dev 2 (très tôt) :**** le champ `Machine` doit être ajouté dans `LogEntry.cs` (projet `EasyLog`) avant que Dev 1 et Dev 2 ne commencent à écrire des logs dans leurs tests – sinon rebase douloureux. Ouvrir une PR sur `develop` dès le premier jour pour ajouter ce champ avec une valeur par défaut vide.

T10 – Documentation et release notes (`feature/v3-release-docs`)

****Fichiers :**** `README.md`, `RELEASE\_NOTES\_v3.0.md`, doc utilisateur (1 page)

- Rédiger `RELEASE\_NOTES\_v3.0.md`.
- Mettre à jour le README : section "Déploiement Docker", nouveaux paramètres, Pause/Stop/Play.
- Mettre à jour la doc utilisateur (1 page max) : nouveaux paramètres, destination logs, pause/stop/play.

> T10 se fait ****en dernier****, une fois que toutes les features sont mergées dans `develop`.

Ordre de merge conseillé dans `develop`

...

main

- └─ develop
 - └─ feature/v3-parallel-execution → Dev 1 (merge en 1er
- base de T5/T6)
 - └─ feature/v3-cryptosoft-singleton → Dev 2 (indépendant,
- merge tôt)
 - └─ feature/v3-docker-log-centralization → Dev 3 (indépendant,

```

champ Machine en 1er)
├── feature/v3-large-file-mutex          → Dev 1  (dépend T1)
├── feature/v3-priority-files            → Dev 1  (dépend T1)
└── feature/v3-settings-ui              → Dev 3  (dépend T2 et
T3 de Dev 1)
├── feature/v3-play-pause-stop          → Dev 2  (dépend T1)
├── feature/v3-business-software-pause → Dev 2  (dépend T5)
└── feature/v3-release-docs             → Dev 3  (merge en
avant-dernier)
└── feature/v3-uml-docs                 → Dev 1  (merge en
dernier)
` ` `

```

Puis créer `release/v3.0` depuis `develop` pour les tests d'intégration finaux.